



INTERNSHIP PROJECT · VERACITIZ

Chronos AI

Watch Price Intelligence

An End-to-End Machine Learning Solution for Real-Time Price Prediction

1,488

Watch Records
Analyzed

6

Regression
Models Evaluated

71%

R^2 Score Achieved

Ready

Production-Ready Deployment

Machine Learning Internship Project · Veracitiz

<https://github.com/Rudra2986/watch-price-prediction>

01 - PROBLEM STATEMENT

The Valuation Challenge

Inconsistent Pricing

Watch prices vary significantly across brand, material, movement type, and design — making manual valuation unreliable and time-consuming.

No Scalable Solution

Existing approaches lack an automated, data-driven method to estimate prices at scale for e-commerce platforms.

Our Goal

Build an ML model that accurately predicts watch prices using historical data — from raw input to **real-time price output**.

Problem

AI Model

Predicted Price

This end-to-end pipeline transforms raw watch attributes into reliable price predictions, eliminating manual effort and inconsistency.

Project Workflow

01

Raw Data

Collect the source watch records and attributes.

03

EDA

Explore trends, patterns, and key relationships.

05

Model Training

Train predictive models on prepared data.

07

Hyperparameter Tuning

Optimize settings to improve prediction quality.

02

Data Cleaning

Remove errors, duplicates, and missing values.

04

Feature Engineering

Transform raw inputs into model-ready signals.

06

Evaluation

Measure accuracy and compare model performance.

08

Deployment

Launch the final model for real-time use.

A streamlined end-to-end workflow that summarizes the full project from raw data to deployment, giving a fast, professional overview at a glance.

Technology Stack

Display the complete technology stack used in the project with modern icons and clean visual cards:



Python

Programming Language



Pandas

Data Manipulation



NumPy

Numerical Computing



Scikit-Learn

ML Preprocessing



CatBoost

Gradient Boosting



Optuna

Hyperparameter Tuning



SHAP

Explainability



Streamlit

Web Deployment



GitHub

Version Control

A modern, production-grade tech stack combining data science, machine learning, and web deployment tools.

Dataset Overview

The **WatchVine E-Commerce Dataset** was scraped and cleaned to support production-grade watch price modeling.

3000+

Raw Columns
Initially scraped, later reduced

25+

Features
Meaningful input variables

1,488

Records
Final cleaned dataset

15

Brands
Distinct watch brands represented

Main feature categories

1	2
Brand Brand identity and positioning	Material Case and strap materials
3	4
Watch Type Analog, digital, or hybrid formats	Category Product classification
5	6
Dial Color Visual style of the watch face	Strap Color Primary strap appearance
7	8
Movement Type Quartz, automatic, or other mechanisms	Case Diameter Size dimension in millimeters
9	10
Water Resistance Durability and usage rating	Is Automatic Binary indicator for automatic movement

The dataset combines brand, design, and technical specifications to create a strong feature set for pricing analysis and predictive modeling.

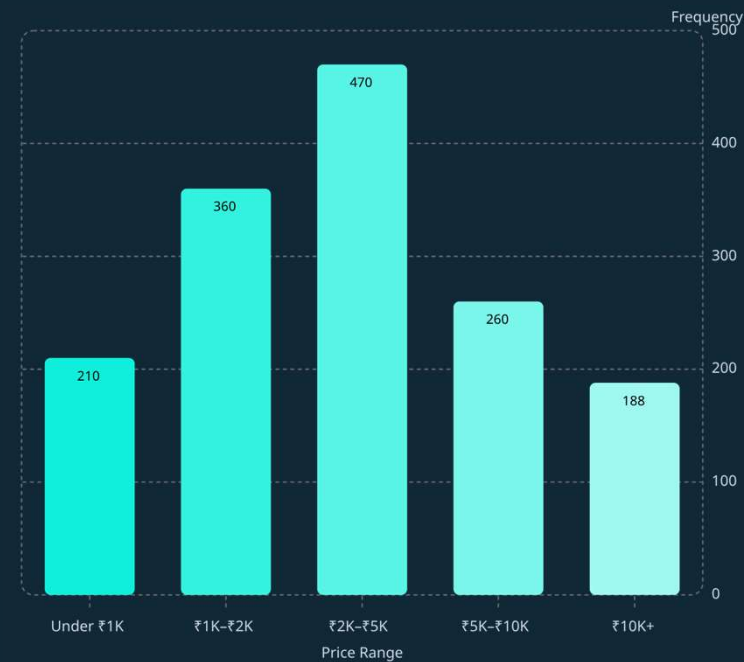
Data Cleaning & Exploratory Data Analysis

Cleaning Pipeline

- Removed 3000+ irrelevant scraped columns
- Converted price strings to numeric INR values
- Handled missing values & standardized text fields
- Removed leakage-prone columns

These steps produced a cleaner, model-ready feature set with consistent formatting and reduced noise.

Price Distribution



The distribution is concentrated in the mid-range, with fewer watches at the premium end, indicating a right-skewed pricing pattern.

Feature Engineering

Feature engineering transformed raw watch data into structured inputs that improved model performance and made patterns easier for the model to learn.

Binary Features

Is Automatic and **Is Luxury Brand** capture key yes/no signals that are easy for the model to interpret.

Categorical Encoding

Brand, **Material**, and **Watch Type** were encoded to convert text categories into usable numeric features.

Numerical Scaling

Case Diameter and **Water Resistance** were scaled to keep numeric features on comparable ranges.

Feature Selection

Redundant and leakage-prone columns were removed to keep only the most informative variables for training.

Encoding Strategy

- One-hot encoding for low-cardinality categories
- Target encoding for higher-cardinality features
- Ordinal handling for ordered numeric-like attributes

Impact

The final feature set was cleaner, more consistent, and more predictive — helping the model learn stronger relationships and achieve better performance.

Machine Learning Model Development

Six regression algorithms were trained and compared using **5-Fold Cross Validation** to identify the best-performing model.

Linear Regression

Baseline model that fits a straight-line relationship between inputs and the target.

Ridge Regression

Regularized linear model that reduces the impact of correlated features.

Random Forest

Ensemble of trees that captures non-linear patterns and interactions.

XGBoost

Gradient boosting method that sequentially improves accuracy.

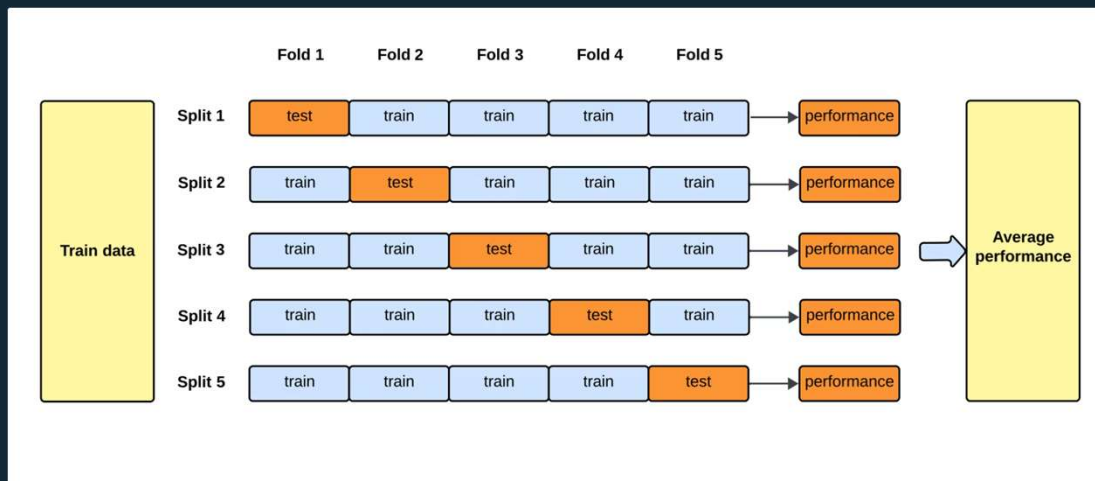
LightGBM

Fast boosting framework designed for efficient training on larger datasets.

CatBoost

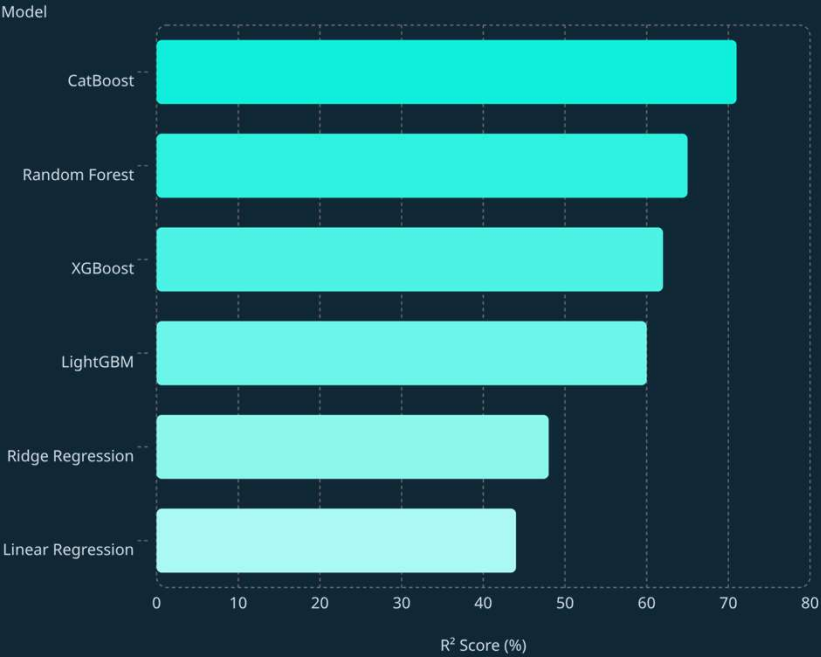
Boosting model that handles categorical features with minimal preprocessing.

5-Fold Cross Validation



This process provides a more reliable estimate of real-world performance by testing each model across multiple data splits. **Final performance = Average of 5 validation scores.**

Model Evaluation & Selection



✔ **Selected as Final Model:** Highest R² score () with superior handling of categorical features and consistent cross-validation performance.

Why CatBoost?

Best Fit

Highest variance explained across all six models.

Categorical Strength

Native handling of categorical features with minimal preprocessing.

Consistent Validation

Strong and stable performance across 5-Fold Cross Validation.

Model Comparison Table

Model Name	R² Score
Linear Regression	44
Ridge	48
Random Forest	65
XGBoost	62
LightGBM	60
CatBoost (Winner)	71

Hyperparameter Tuning & Explainable AI

Optuna — 100 Trials

Automated Bayesian optimization to find the best CatBoost setup.

Iterations

540

Depth

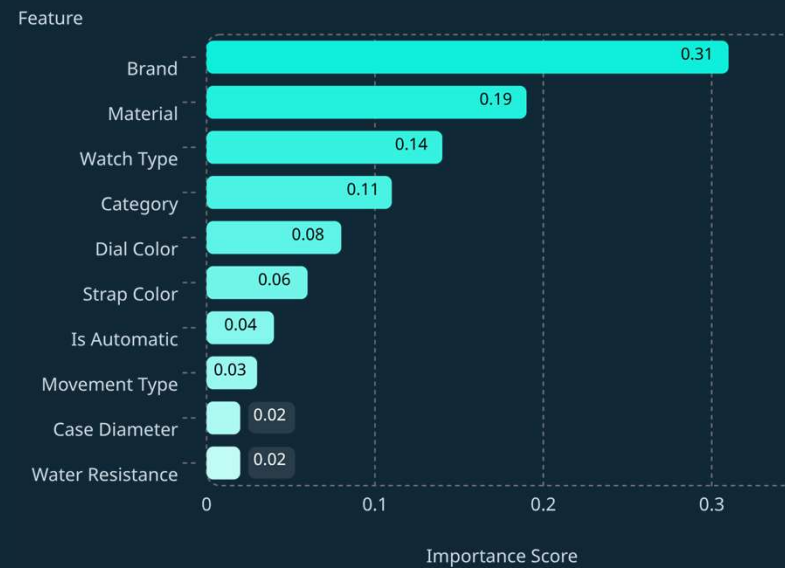
5

Learning Rate

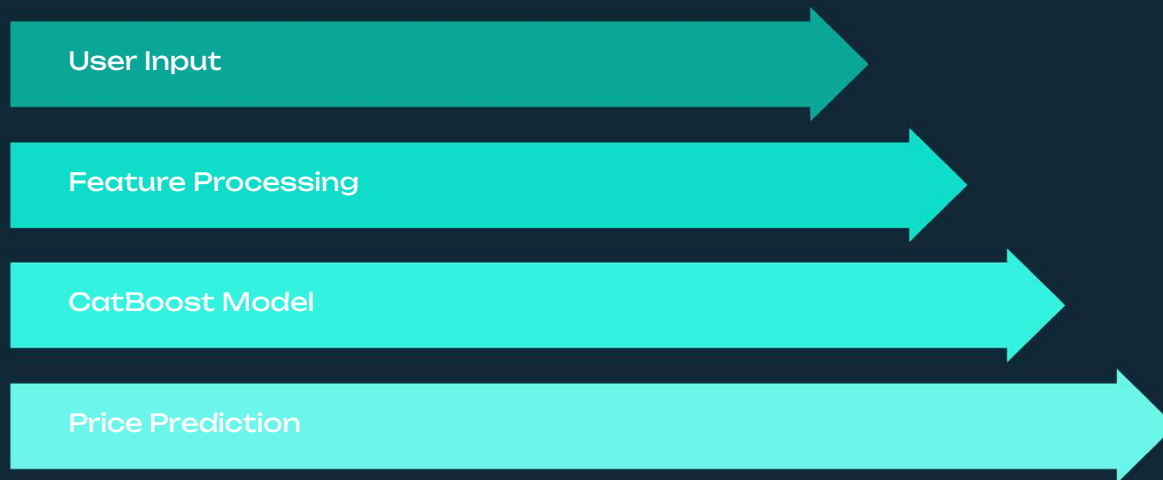
0.0465

SHAP Explainability

SHAP values reveal which features most influence price predictions, enabling model interpretability and trust.

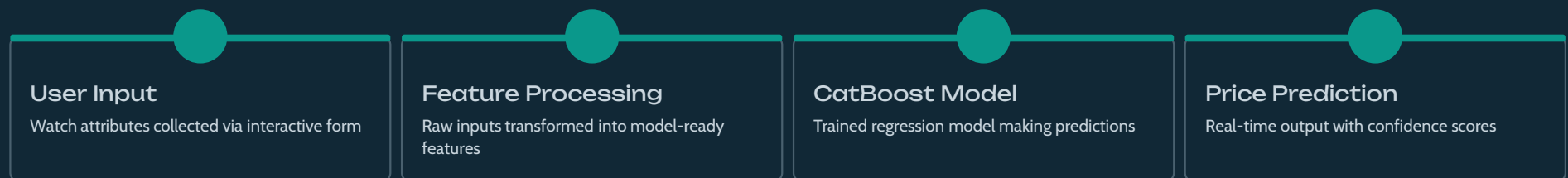


Deployment Architecture



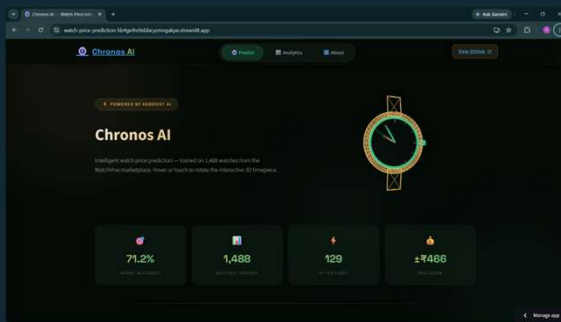
From user input to price output in seconds — the Chronos AI pipeline delivers production-ready predictions through a clean, intuitive interface.

Architecture Overview



09 · LIVE APPLICATION

Chronos AI — Live Application



Home Page

App overview and navigation

The screenshot shows the prediction page of the Chronos AI application. It is titled 'Configure Watch Attributes'. There are two columns of dropdown menus for selecting watch attributes: 'Company' (Men's Watch), 'Brand' (Armani), 'Watch Type' (Luxury), 'Style' (Luxury), 'Movement' (Automatic), 'Case Type' (Metal Mesh), 'Case Material' (Carbon Fiber), 'Strap Material' (Leather), 'Dial Color' (Black And Gold), and 'Strap Color' (Black And Gold). At the bottom, there is a 'Candidate Price Prediction' bar.

Prediction Page

Enter watch attributes



Results Page

Price output and explanations

Chronos AI is deployed as a production-ready Streamlit web application, enabling real-time watch price predictions with explainable AI insights.

Results, Challenges & Future Scope

71%

R² Score

Final CatBoost model performance

₹466

RMSE

Root Mean Squared Error

₹294

MAE

Mean Absolute Error

0.07%

Brand Unknown

— critical fix

Business Impact

Automated watch valuation

Reduced manual price estimation effort

Scalable for e-commerce platforms

Real-time prediction capability

Production-ready deployment

🏆 Challenge Solved

Brand Sparsity Problem:

87.5% → 0.07%

Unknown brands reduced through intelligent feature engineering, driving significant model performance gains.

🔮 Future Scope

- Larger, more diverse datasets
- Image-based watch analysis (CNN/viT)
- Text embeddings from product descriptions
- Automated retraining pipeline
- Cloud deployment (AWS / GCP)

Successfully built an end-to-end Machine Learning solution — from raw scraped data to production-ready deployment. **Chronos AI is live.**

11 · KEY LEARNINGS

Key Learnings & Achievements

This project provided comprehensive experience across the machine learning lifecycle, highlighting mastery in:

End-to-End Machine Learning Pipeline

Data Cleaning & EDA

Feature Engineering

Regression Modeling

Achievement Badges



End-to-End ML Pipeline



Explainable AI



Hyperparameter Optimization



Production Deployment

This project demonstrates mastery of the complete machine learning lifecycle, from data acquisition to production deployment.

Thank You

Questions & Discussion Welcome

Rudra Patel · Machine Learning Intern · Veracitiz

Project Links & Resources

GitHub Repository

Complete source code, notebooks, and documentation

[GitHub - Rudra2986/watch-price-prediction: Watch price prediction model using ML](#)

Live Application

Try Chronos AI in action

[Chronos AI — Watch Price Intelligence](#)

GitHub Repository QR Code



Live Application QR Code



Scan to access the project or visit the links above